

Lizard Bot - Software Requirements Document

Version 1.0

Revision History

Version	Date	Author	Description
1.0	2025-09-15	David Caldwell	Initial release of the Software Requirements Document

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for the Lizard Bot Discord application. The bot monitors Discord text channels and provides automated responses to specific keyword triggers.

1.2 Scope

The Lizard Bot will operate as a single-instance Discord application that:

- *Monitors multiple text channels simultaneously via persistent WebSocket connection*
- *Detects specific keywords*
- *Provides automated responses through direct Discord REST API calls*
- *Functions across multiple Discord servers with high reliability*
- *Maintains persistent operation with process monitoring and auto-restart*

1.3 Definitions & Abbreviations

- **Bot:** *Discord application/bot user account*
- **Guild:** *Discord server*
- **Channel:** *Discord text or voice channel*
- **API:** *Application Programming Interface*

- *MVP: Minimum Viable Product*
- *EC2: Amazon Elastic Compute Cloud*

2. System Architecture Requirements

2.1 Modular Application Design (AR-001)

Requirement: System functionality must be logically organized within modular components of a single application.

Acceptance Criteria:

- *Message processing pipeline maintains clear internal separation of concerns*
- *Configuration management is handled by dedicated module*
- *Discord Gateway and REST API interactions are isolated in separate modules*
- *System can recover from individual module failures without full application restart*

2.2 Event-Driven Processing (AR-002)

Requirement: Application must handle operations.

Acceptance Criteria:

- *Message processing uses non-blocking operations*
- *Multiple Discord events handled asynchronously*
- *Failed operations handling without blocking other operations*
- *System remains responsive during high message volume*
- *Long-running operations are properly queued*

3. Functional Requirements

3.1 Core Message Processing (FR-001)

Requirement: The system must monitor all accessible text channels for incoming messages.

Acceptance Criteria:

- *Receives message events from Discord Gateway in real-time*
- *Message processing occurs within 200ms of receipt*
- *System ignores its own messages to prevent loops*
- *Handles messages with embeds, attachments, and mentions*
- *Maintains persistent WebSocket connection with automatic reconnection logic*

3.2 Keyword Detection (FR-002)

Requirement: *The system must detect configured keywords in messages.*

Acceptance Criteria:

- *Detection is case-insensitive ("lizard", "LIZARD", "Lizard" all trigger)*
- *Word boundary detection (matches "lizard" but not "lizards")*
- *Detection works with punctuation ("lizard!", "lizard.", "lizard?")*
- *Supports configurable keywords from local storage with hot-reload capability*

3.3 Automated Response (FR-003)

Requirement: *The system must send responses when keywords are detected.*

Acceptance Criteria:

- *Response sent to the same channel as trigger message*
- *Response appears within 500ms of trigger detection*
- *Implements 3-second cooldown per channel*
- *Respects Discord API rate limits with built-in throttling*

3.4 Multi-Guild Support (FR-004)

Requirement: *The bot must operate across multiple Discord servers simultaneously from single instance.*

Acceptance Criteria:

- *Single WebSocket connection handles multiple guild events*
- *Guild configurations stored in local SQLite database*
- *Independent cooldowns per guild/channel maintained in memory*
- *Supports up to 100 guilds in production on single instance*

3.5 Configuration Management (FR-005)

Requirement: *Bot configuration must be manageable without application restart.*

Acceptance Criteria:

- *Keywords stored in local SQLite database*
- *Response messages configurable per keyword with hot-reload*
- *Guild-specific settings supported through local configuration*
- *Configuration changes take effect within 30 seconds*
- *Configuration backup and versioning through file system*

4. Non-Functional Requirements

4.1 Performance (NFR-001)

- **Response Time:** <200ms average processing time (in-memory processing)
- **Throughput:** 200-500 messages per second (single instance limitation)
- **Startup Time:** <15 seconds for full application initialization
- **Database Latency:** <5ms for local SQLite operations
- **Memory Usage:** <200MB RAM under normal operation
- **CPU Usage:** <50% under normal load

4.2 Reliability (NFR-002)

- **Uptime:** 99.5% availability target for single instance
- **Error Recovery:** Automatic process restart
- **Connection Recovery:** Automatic Discord reconnection with exponential backoff
- **Graceful Degradation:** Continue operation during partial failures
- **Data Persistence:** Local SQLite database survives application restarts

4.3 Scalability (NFR-003)

- **Vertical Scaling:** Instance can be upgraded for increased capacity
- **Local Database:** SQLite scales to moderate data volumes (up to 100GB)
- **Buffer Management:** In-memory queuing for message burst handling
- **Future Horizontal:** Architecture supports multiple instance deployment
- **Resource Efficiency:** Optimized memory usage with garbage collection tuning

4.4 Security (NFR-004)

- **Token Security:** Discord bot token stored in environment variables
- **Access Control:** EC2 instance security groups restrict network access
- **Network Security:** HTTPS/WSS for all Discord communications
- **Encryption:** TLS for all external communications
- **Secrets Management:** Environment-based secret handling

4.5 Observability (NFR-005)

- **Logging:** Comprehensive Winston-based logging with rotation
- **Metrics:** Application-level metrics collection
- **Monitoring:** Process health monitoring
- **Alerting:** Automated alerts for application failures
- **Debugging:** Structured logging for troubleshooting

5. Data Requirements

5.1 Configuration Storage

- **Purpose:** Store bot configuration and keywords locally
- **Requirements:**
 - SQLite database for local storage with ACID compliance
 - JSON configuration files for environment-specific settings
 - File-based backup to S3 for disaster recovery
 - Hot-reload capability for configuration changes

5.2 Cooldown Tracking

- **Purpose:** Track per-channel response cooldowns in-memory
- **Requirements:**
 - In-memory Map structure with automatic cleanup
 - backup for persistence across restarts
 - Sub-millisecond read/write performance
 - Channel and guild-specific tracking with TTL cleanup

5.3 Analytics Storage

- **Purpose:** Store usage metrics and statistics locally
- **Requirements:**
 - Simple metrics storage with periodic export
 - Basic query capabilities for operational data
 - Guild and channel metrics with manual aggregation

5.4 Log Storage

- **Purpose:** Application logs and audit trails
- **Requirements:**
 - Centralized logging from single application instance
 - Local log files with rotation and archival
 - Configurable retention periods

5.5 Secret Storage

- **Purpose:** Secure storage of sensitive configuration
- **Requirements:**
 - Environment variables for bot tokens
 - file encryption for sensitive config
 - Secure environment variable handling

6. Integration Requirements

6.1 Discord API Integration (IR-001)

- **Gateway:** Single persistent WebSocket connection for real-time events
- **REST API:** HTTP client for sending messages and API calls
- **API Version:** Discord API v10
- **Intents:** MESSAGE_CONTENT and GUILD_MESSAGES
- **Rate Limiting:** Built-in rate limiting with exponential backoff

7. Future Enhancement Requirements

7.1 Phase 2 Features

- **Web Dashboard:** Express.js web interface for configuration
- **Custom Commands:** Slash commands for bot interaction
- **Multiple Keywords:** Support arrays of trigger words
- **Regex Patterns:** Advanced pattern matching capabilities
- **Guild Administration:** Per-guild configuration through web interface

7.2 Phase 3 Features

- **Voice Support:** Text-to-speech in voice channels
- **AI Integration:** Natural language response generation
- **Horizontal Scaling:** Multi-instance deployment with load balancing
- **Advanced Analytics:** Time-series database integration
- **Multi-region:** Geographic distribution for lower latency

8. Acceptance Criteria

8.1 MVP Completion

- ☐ Single application receiving Discord events
- ☐ Keyword detection functional with "lizard" trigger
- ☐ Response sending working correctly
- ☐ database operational for configuration
- ☐ 3-second cooldown implemented with in-memory tracking
- ☐ Multi-guild support verified
- ☐ Automatic reconnection logic tested

8.2 Production Readiness

- ☐ Load testing completed for single instance

- *[] Error handling and recovery tested*
- *[] Documentation complete*
- *[] Operational runbook created*

9. Constraints & Assumptions

9.1 Technical Constraints

- *Discord API rate limits (50 requests per second per bot token)*
- *Single instance CPU and memory limitations (t4g.nano)*
- *Network bandwidth constraints (EC2 instance burst limits)*
- *Local SQLite database size limitations (<100GB practical)*
- *Single point of failure for hardware issues*

9.2 Business Constraints

- *AWS service limits and quotas*
- *Discord Terms of Service compliance*
- *Budget constraints (\$3-5/month maximum)*
- *Development timeline and resource availability*
- *Maintenance and operational overhead*

9.3 Assumptions

- *Discord API remains stable and backwards compatible*
- *Network connectivity remains stable*
- *Bot token remains valid and secure*

This document represents the complete technical requirements for the Lizard Bot. It defines what the system must do while optimizing for simplicity, cost-effectiveness, and maintainability.